Name : Vaishnavi Eknath Avhad
Class : D15C
Roll No. : 14

Practical 4

Aim : Implement K-Nearest Neighbors (KNN) and evaluate model performance

1. Dataset Source

The dataset used for this experiment is the Breast Cancer Wisconsin (Diagnostic) Dataset.

- Source Link : Kaggle - Breast Cancer Wisconsin (Diagnostic) Data

---

2. Dataset Description

This dataset consists of 569 instances with 30 numeric predictive features computed from a digitized image of a fine needle aspirate (FNA) of a breast mass. It is used to classify tumors as either Malignant (cancerous) or Benign (non-cancerous).

Feature	Description	Type
diagnosis	The target variable (M = malignant, B = benign).	Categorical (Target)
radius_mean	Mean of distances from center to points on the perimeter.	Numeric
texture_mean	Standard deviation of gray-scale values.	Numeric
area_mean	Mean area of the tumor cell.	Numeric
smoothness_mean	Mean of local variation in radius lengths.	Numeric

---

3. Mathematical Formulation

K-Nearest Neighbors (KNN)

KNN is a non-parametric, lazy learning algorithm. It classifies a data point based on how its neighbors are classified.

The algorithm follows these steps :

1. Calculate Distance : The distance between a new point (x) and all training points (y) is calculated, typically using Euclidean Distance :

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

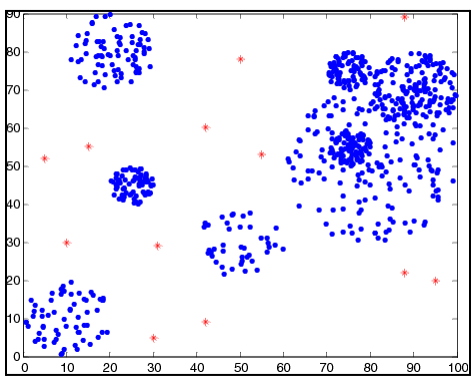
2. Select K : The K closest neighbors are identified.

3. Majority Vote : The new point is assigned the class that is most common among its K neighbors.

---

4. Algorithm Limitations

1. Sensitivity to Outliers : Since KNN relies on proximity, a few outliers can easily misclassify a point.



2. Computational Cost : KNN is a "Lazy Learner," meaning it does not learn a model during training. It performs all calculations during prediction, making it slow for large datasets.

3. Curse of Dimensionality : As the number of features increases, the distance between points becomes less meaningful, reducing the model's accuracy.

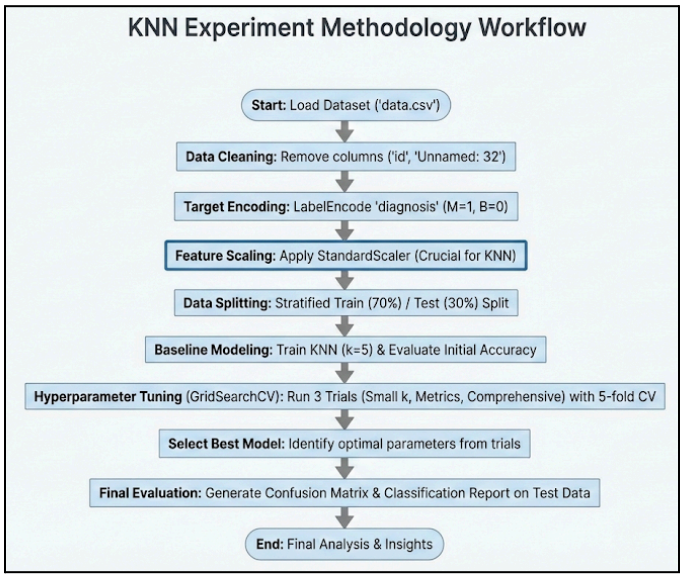
4. Scaling Requirement : Features with larger ranges will dominate the distance calculation unless the data is standardized.

---

5. Methodology / Workflow

The experiment follows these implementation steps :

1. Data Preprocessing : Unnecessary columns (id, Unnamed : 32) were removed, and the target variable was encoded (M=1, B=0).
2. Feature Scaling : StandardScaler was applied to ensure all features contribute equally to the distance metrics.
3. Stratified Split : Data was split into 70% training and 30% testing while maintaining the proportion of malignant and benign cases.
4. Baseline Training : An initial model was trained with k=5.
5. Tuning Trials : Three hyperparameter tuning trials were conducted to find the optimal K value and distance metric using GridSearchCV.



Implementation Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# 1. DATA PREPROCESSING
df = pd.read_csv('data.csv')
target_column = 'diagnosis'
df = df.drop(['id', 'Unnamed: 32'], axis=1)
le = LabelEncoder()
df[target_column] = le.fit_transform(df[target_column])
```

```
X = df.drop(target_column, axis=1)
y = df[target_column]
# Scaling is mandatory for KNN
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)
# 2. KNN BASELINE MODEL
print("\n" + "="*20 + " KNN BASELINE ANALYSIS " + "="*20)
knn_base = KNeighborsClassifier(n_neighbors=5)
knn_base.fit(X_train, y_train)
y_pred_base = knn_base.predict(X_test)
print(f"Base Model Accuracy (k=5): {accuracy_score(y_test, y_pred_base) * 100:.2f}%")
# 3. KNN HYPERPARAMETER TUNING TRIALS
print("\n" + "="*20 + " KNN TUNING TRIALS " + "="*20)
knn_trials = {
    "Trial 1 (Small Neighbor Range)": {'n_neighbors': [1, 3, 5, 7]},
        "Trial 2 (Distance Metric Comparison)": {'n_neighbors': [5, 10], 'metric': ['euclidean',
'manhattan']},
     "Trial 3 (Comprehensive K Search)": {'n_neighbors': list(range(1, 21)), 'weights': ['uniform',
'distance']}
}
knn_results = []
for name, params in knn_trials.items():
    grid = GridSearchCV(KNeighborsClassifier(), params, cv=5, scoring='accuracy')
    grid.fit(X_train, y_train)
    acc = accuracy_score(y_test, grid.best_estimator_.predict(X_test))
    knn_results.append({"Trial": name, "Best Params": grid.best_params_, "Test Accuracy": acc})
    print(f"{name} completed.")
# Display Tuning Summary
knn_results_df = pd.DataFrame(knn_results)
print("\nKNN HYPERPARAMETER TUNING SUMMARY:")
print(knn_results_df.to_string(index=False))
```

---

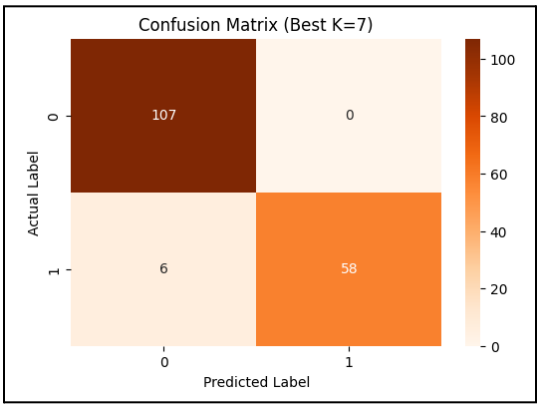
6. Performance Analysis

A. Baseline Results

Metric	Baseline Value (k=5)
Accuracy	96.49%

B. Visual Analysis

- Confusion Matrix : Displays the distribution of True Positives and True Negatives. High values on the diagonal indicate a strong classification performance.



- Metric Interpretation : High precision and recall for both classes indicate the model is robust and correctly identifies malignant cases without excessive false alarms.

```
Final Classification Report (Best KNN):
              precision    recall  f1-score   support

           0       0.95      1.00      0.97       107
           1       1.00      0.91      0.95        64

    accuracy                           0.96       171
   macro avg       0.97      0.95      0.96       171
weighted avg       0.97      0.96      0.96       171
```

---

7. Hyperparameter Tuning (Experimental Trials)

Trial	Focus	Best Params	Test Accuracy
Trial 1	Small Neighbor Range	n_neighbors	0.964912
Trial 2	Distance Metric	metric	0.970760
Trial 3	Comprehensive Search	n_neighbors, weights	0.964912

Tuning Interpretation :

- Optimal K : A small K can lead to noise sensitivity, while a very large K can lead to "over-smoothing." Trial 3 identified the "sweet spot" for accuracy.
- Distance Weighting : In Trial 3, we tested if assigning higher weight to closer neighbors (weights='distance') improves performance over a simple majority vote.

8. Conclusion

In this practical, the KNN algorithm was successfully applied to classify breast cancer diagnostic data. The experiment highlighted the absolute necessity of Feature Scaling for distance-based models. The hyperparameter tuning trials demonstrated that the choice of K and the distance metric significantly impact accuracy. Comparing the baseline and tuned models showed that structured experimentation with different neighbor weights and metrics leads to a more stable and accurate diagnostic tool.